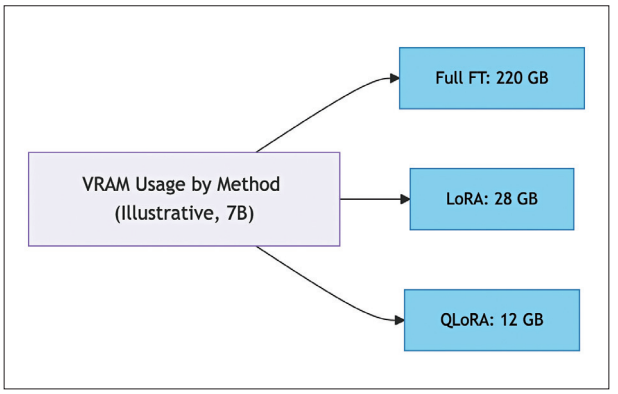


The analogy has limits: adapters cannot fix fundamental gaps in the base model's reasoning ability.

QLoRA combines LoRA-style adapter training with a quantised, commonly four-bit, base model. The compressed base occupies less memory while the additional adapter weights remain trainable. This can make larger checkpoints accessible on more limited GPUs, although sequence length, optimiser, activations and platform support still determine whether training actually fits. QLoRA is not the same as opening the GGUF file you used for inference and clicking Train. We will return to its setup after completing an ordinary LoRA run.

Design the lesson before the training run

Use the cleaned-data discipline from Chapter 2. We will teach our model to turn informal laptop-support requests into one consistent JSON object. Define the allowed categories as `warranty`, `returns`, `repair` and `other`; urgency as `low`, `medium` or `high`. Each answer must contain exactly `category`, `urgency` and `next_action` and must avoid inventing policy details.

Write examples covering genuinely different language: a cracked hinge, a failed charging port, an opened-box return, missing proof of purchase, a Hindi-English complaint and a message mentioning two unrelated issues. Make the expected next action specific but safe: "Request invoice and model number," not "Approve a replacement." A complaint about possible battery swelling should lead to an appropriate safety-focused escalation rather than unsupported troubleshooting.

Save training examples as JSONL records with a `prompt` and `completion`. For TRL's conversational prompt-completion format, one record looks like this:

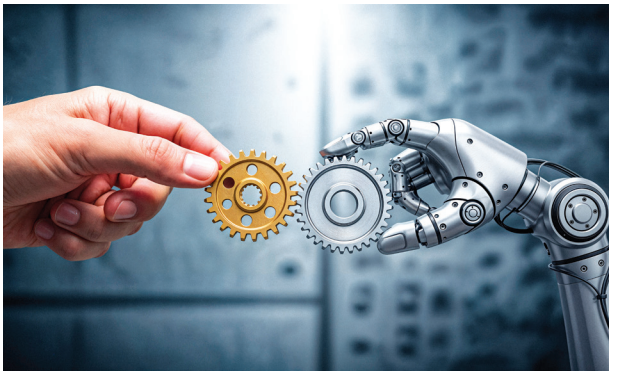
{"prompt":[{"role":"user","content":"My laptop charger has stopped working."}],"completion":[{"role":"assistant","content":"{\"category\":\"repair\",\"urgency\":\"medium\",\"next_action\":\"Request model and warranty details\"}"}]}

Build a small scoring rubric before training. For every test complaint, record the correct category, urgency and acceptable next action. Award one point for each correct field, and track an additional failure whenever the output is not parseable JSON. This prevents a cheerful but noncompliant answer from receiving a pass. Store the ground truth separately from the dataset the trainer sees. Set an explicit rule for ambiguous cases: ask for the missing evidence rather than guessing eligibility.

Keep instructions consistent across examples; otherwise the model may learn conflicting rules. Prepare perhaps 80–150 diverse, human-reviewed records for an initial experiment, not because that number guarantees quality, but because it is small enough to inspect. Split by customer scenario into training and validation files; keep an independent test set untouched. Do not put paraphrases of the same complaint on both sides of the split. Measure performance on Indian English and code-mixed examples rather than assuming competence will transfer automatically.

Prepare the training environment

Return to `my-ai`, activate its virtual environment and verify that Python can access a compatible accelerator. An NVIDIA GPU with supported CUDA software is the simplest path for this demonstration; CPU training may be extremely slow. Other accelerators can work with different configurations, but do not assume identical support for every quantisation library.



Install a mutually compatible PyTorch build following the official instructions for your platform, then install the remaining packages:

```
pip install -U transformers datasets accelerate peft trl
```

Use a separate environment if upgrading these libraries would break the working inference setup from Part 1. Save package versions after your first successful run. For this exercise we select `Qwen/Qwen2.5-0.5B-Instruct`, a small publicly available, Apache-2.0-licensed checkpoint. Its size makes it useful for teaching the workflow, not a promise of production-grade judgment.

One crucial difference from October's book: the quantised GGUF you may have downloaded for Ollama or llama.cpp is an inference artefact. Our training script loads the corresponding